OS ASSIGNMENT - 4

Name:SHRUTI BURUKULE

SRN:PES1UG24CS625

PHASE 1:

1A)

```
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
shru@ubuntu:~/PES1UG24CS625-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B)

```
shru@ubuntu:~/PES1UG24CS625-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
shru@ubuntu:~/PES1UG24CS625-pes-vcs$
```

PHASE 2:

2A)

```
shru@ubuntu:~/PES1UG24CS625-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
shru@ubuntu:~/PES1UG24CS625-pes-vcs$
```

2B)

```
All Phase 2 tests passed.
shru@ubuntu:~/PES1UG24CS625-pes-vcs$ find .pes/objects -type f
shru@ubuntu:~/PES1UG24CS625-pes-vcs$
```

PHASE 3:

3A)

```
shru@ubuntu:~/PES1UG24CS625-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

3B)

```
shru@ubuntu:~/PES1UG24CS625-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776319325 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776319336 6 file2.txt
shru@ubuntu:~/PES1UG24CS625-pes-vcs$
```

PHASE 4:

4A)

```
shru@ubuntu:~/PES1UG24CS625-pes-vcs$ ./pes log
commit ed8f2eed58b4f2eca07452da4f933171273bea203483d0d42e17779d28f4a272
Author: Your Name <PES1UG24CS625>
Date:   1776320938

    Add farewell

commit c4d062a2c2e0e184334bb0827e74c9a8b1857c221de2fae2ea09fe02d7215464
Author: Your Name <PES1UG24CS625>
Date:   1776320928

    Add worldell

commit eae795284c860d51c8b5b8202e45487ba930517dc2f45e02db809b0d4605b486
Author: Your Name <PES1UG24CS625>
Date:   1776320845

    Initial committial comQ
```

4B)

```
shru@ubuntu:~/PES1UG24CS625-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/c4/d062a2c2e0e184334bb0827e74c9a8b1857c221de2fae2ea09fe02d7215464
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ea/e795284c860d51c8b5b8202e45487ba930517dc2f45e02db809b0d4605b486
.pes/objects/ed/8f2eed58b4f2eca07452da4f933171273bea203483d0d42e17779d28f4a272
.pes/refs/heads/main
shru@ubuntu:~/PES1UG24CS625-pes-vcs$ cat .pes/refs/heads/main
```

4C)

```
.pes/refs/heads/main
shru@ubuntu:~/PES1UG24CS625-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
ed8f2eed58b4f2eca07452da4f933171273bea203483d0d42e17779d28f4a272
ref: refs/heads/main
```

Q5.1

To implement pes checkout <branch>, first read the commit hash stored in .pes/refs/heads/<branch>. Then update .pes/HEAD so that it points to that branch.

After that, load the commit object and its tree object. The working directory should then be changed to match that tree.

This means files present in the target branch should be created, files not present in the target branch should be deleted, and existing files should be replaced with the correct version. The .pes/index file should also be updated to match the checked-out branch.

This operation becomes difficult because some files may already be modified by the user, some may be deleted, and there may also be untracked files in the working directory. Checkout should not overwrite such files directly.

Q5.2

To detect uncommitted changes, compare the current working directory files with the entries stored in .pes/index.

For each tracked file, read the file from the working directory, compute its blob hash, and compare it with the hash stored in the index. If the hashes are different, then the file has been modified after staging.

Then compare the file in the current branch with the file in the target branch.

If the working file is modified and the same file is different in the target branch, then checkout should stop because switching branches would overwrite the user's changes.

Q5.3

Detached HEAD means .pes/HEAD stores a commit hash directly instead of storing something like ref: refs/heads/main.

If a commit is made in this state, the new commit is created normally, but no branch points to it.

Because of that, those commits can become unreachable later.

The user can recover them by taking the commit hash and creating a new branch pointing to it. After that, HEAD can be updated to point to that branch

Q6.1

Garbage collection can be done using two steps: mark and sweep.

In the mark step, start from all branch references inside .pes/refs/heads/. Read their commit hashes and mark all reachable commits. From each commit, mark its parent commits and tree object. From tree objects, mark all subtrees and blobs.

A hash set can be used to store reachable hashes because searching is fast.

In the sweep step, go through every object inside .pes/objects/. If its hash is not in the reachable set, delete it.

For 100,000 commits and 50 branches, the algorithm may need to visit around 100,000 commit objects, around 100,000 tree objects, and many blob objects. So overall it may need to check several lakh objects.

Q6.2

Running garbage collection during a commit can be dangerous.

Suppose a commit is being created. First the blob object is written, then the tree object is written, then the commit object is written, and finally the branch reference is updated.

If garbage collection runs in between, it may see the new blob object as unreachable because no commit points to it yet.

So GC may delete that blob.

Later, the commit process may create a tree object pointing to that deleted blob, causing corruption.

Git avoids this by using locks, temporary files, delayed deletion, and by not deleting very recent unreachable objects immediately.